

Tool 体系：原理解析、实现与代码讲解

Provider 让 Agent 会调模型，ReAct 让它会思考，但到现在它还只会"想"和"说"。这节讲的 Tool，是让 Agent 真正能动手干事的那双手。四件事照旧：Tool 是什么、动手前该想清楚什么、代码怎么写、怎么用和怎么验。

技术栈还是 JDK 21 + Spring Boot 3.x，配合 Spring AI 的 @Tool 注解和 MCP 协议。下面的代码是示意。

一、Tool 是什么，干嘛用的

一句话：**Tool 就是 Agent 的手——LLM 负责想，Tool 负责真的去读文件、跑命令、调接口。**

大模型本身只会生成文字，它读不了你磁盘上的文件，也发不出一个 HTTP 请求。想让 Agent 真的干活，就得给它一批能操作外部世界的工具。**LLM 决定"调哪个工具、传什么参数"（靠 16 节说过的 Function Calling），OryxOS 负责把这个工具真正执行掉、再把结果递回给 LLM。**这正是 ReAct 循环里 "Act" 那一步干的事。



OryxOS 的 Tool 分两类：

- **内置 Tool**：OryxOS 自带的基础工具，核心阶段一共九个——`read_file` / `write_file` / `list_dir`（文件，3 个）、`shell`（跑命令，1 个）、`http_get` / `http_post`（发请求，2 个）、Memory 用的 `save_memory` / `recall_memory`（2 个），再加上 19 节刚讲过的 `notify`（推送，1 个）。它们覆盖"读写文件、跑命令、调 API、记事、往外推通知"这条最短链路。（对齐业界主流 Agent 的实践，实现时又补了 `edit_file` / `grep` / `glob` / `ask_user` / `web_search` 五个基础工具，见本节第六部分。）
- **Plugin Tool**：业务方自己扩展的工具。企业要做运维助手、客服助手，靠的就是它——OryxOS 本身只给基础工具，真正的业务能力是业务方接进来的。

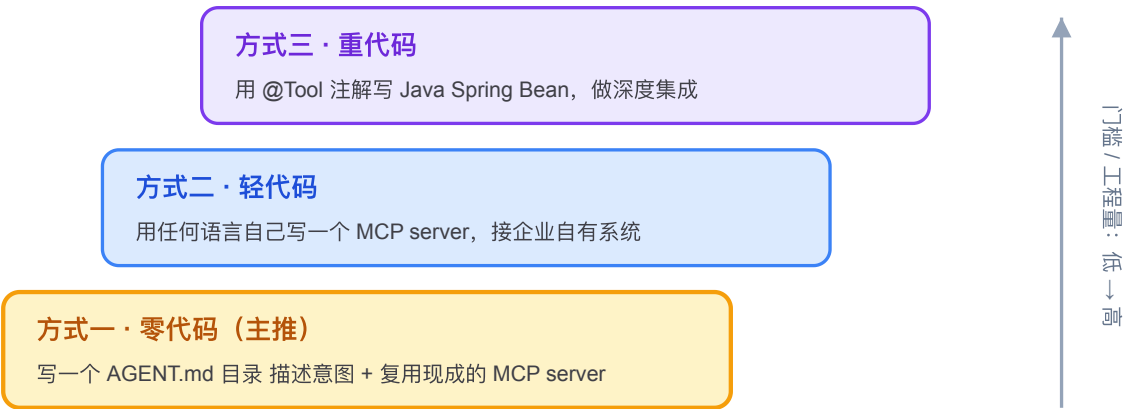
这一节的重点，就在 Plugin Tool 怎么设计。

二、动手前先想清楚几件事

第一，先定一个统一的工具抽象，屏蔽掉"来源"。内置的、业务方用 Java 写的、通过外部服务接进来的，工具来源五花八门。如果 ReAct 循环要分来源区别对待，代码会很快乱掉。所以先定一个统一接口 `OryxTool`：不管工具从哪来，都包装成 `OryxTool` 注册进一个 `ToolRegistry`，**ReAct 循环只跟 `OryxTool` 打交道，完全不感知它背后是什么**。这个抽象是整个 Tool 体系的地基，得第一个定下来。

动手前先检查一下现有代码：如果 `OryxTool` 接口是前面几周就写好的，先确认它有没有 `getInputSchema()` 这个方法——早期版本经常只顾着 `getName / getDescription / execute` 三个"看得见"的方法，把 `schema` 这块漏掉。这个方法不补上，`Provider` 那边想把工具翻译成 `Function Calling` 格式时，根本没地方拿参数说明，这一步会直接卡住。

第二，**Plugin Tool 给三档接入方式，门槛从低到高**。不同的人扩展能力的水平不一样，所以给三条路，让业务方按需选：



选择标准就一句话：**能用方式一就不用方式二，能用方式二就不用方式三**。因为方式一最优雅——业务方只描述"想干什么"，具体调哪个工具、怎么组合，交给 LLM 自己想。比如"每天早上把昨天的 GitHub PR 进度推到 Slack"，写一份 markdown、复用现成的 `github-mcp` 和 `slack-mcp`，一行代码都不用写。

第三，所有工具执行前都要过安全校验。工具能读文件、跑命令、发请求，一旦被乱调就是事故。核心阶段的办法是应用层白名单：文件操作查路径白名单、Shell 查命令白名单、HTTP 查域名白名单，执行前统一过一层校验，校验不过直接拦下——这层校验具体怎么设计是 23、24 节 `Sandbox` 模块的内容，这里先知道执行链路里有这一步。这是核心阶段唯一的 Tool 治理手段，得在设计时就串进执行链路里。

想清楚就这几句：先立 `OryxTool` 统一抽象、Plugin 给三档接入、执行前一律过白名单。

三、代码怎么写

Tool 相关的东西核心阶段合成一个模块（内置 Tool、MCP Client、ToolRegistry、安全校验都在里面，具体实现见 23、24 节），因为它们共享同一个 `OryxTool` 抽象，没必要拆细。

先看统一抽象 **OryxTool**。它约定四个方法，任何来源的工具都得实现：

```
public interface OryxTool {
    String getName();           // 工具名，LLM 靠它点名要调谁
    String getDescription();    // 干什么用的，给 LLM 看
    JsonSchema getInputSchema(); // 参数长什么样（就是 16 节说的 schema）
    ToolResult execute(JsonNode input); // 真正执行，输入 JSON，返回结果
}
```

`ToolResult` 里带四样东西：成功标识、结果内容、错误信息、是否可重试。为什么要“是否可重试”？因为 ReAct 循环拿到失败结果时，得知道这错值不值得再调一次。

有了这个接口，三种来源的工具就有了统一的样子：

- 内置 Tool 和方式三（Java `@Tool`）：用 Spring AI 的 `@Tool` 注解标在 Java 方法上，启动时自动扫描、生成 schema、包装成 `OryxTool`。
- 方式二（MCP）：`McpClientService` 连上外部 MCP server，把它暴露的工具也包装成 `OryxTool`。这两个类的实现规格如下（31 节的日报 Agent 硬依赖它，不能只停在概念）：

```
@Component
public class McpClientService {

    @PostConstruct
    public void connectAll() {
        for (McpServerConfig cfg : loadConfigs()) { // 读
            .oryxos/mcp_servers.yaml
            try {
                McpClient client = McpClient.connect(cfg); // stdio 或
                // SSE
                for (McpToolSpec spec : client.listTools()) { // 调
                    // tools/list
                    toolRegistry.register(new McpToolAdapter(client, spec));
                    // 包装注册
                }
            } catch (Exception e) {
                log.warn("MCP server {} 连接失败，跳过它的工具", cfg.name(), e);
                // 外部依赖失联不拖垮自身启动一只 WARN，OryxOS 照常起
            }
        }
    }
}
```

```

    }
}

```

```

public class McpToolAdapter implements OryxTool {
    // getName/getDescription/getInputSchema 直接映射 MCP 的 tools/list 返回
    @Override
    public ToolResult execute(JsonNode input) {
        return client.callTool(spec.name(), input) // JSON-RPC 转发给
MCP server

        .map(ToolResult::success)
        .orElseGet(() -> ToolResult.failure("MCP 调用失败", true));
    }
// 可重试
}
}

```

关键就两条：**注册时**把每个 MCP 工具包装成 `OryxTool` 塞进同一个 `ToolRegistry` (`ReAct` 循环由此对来源无感知)；**执行时**通过 MCP 协议原样转发、结果包成 `ToolResult`。失联的 server 记 WARN 跳过，不能变成自己的启动故障。

一个内置 **Tool** 长什么样。拿 `http_get` 举例：

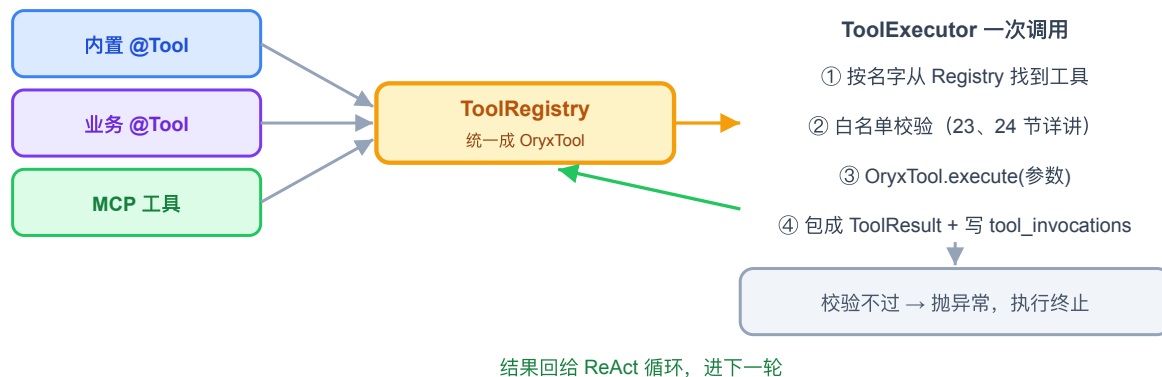
```

@Tool(name = "http_get", description = "发起一个 HTTP GET 请求, 返回响应体")
public String httpGet(@ToolParam("要请求的完整 URL") String url) {
    sandbox.enforce(new SandboxAction(ActionType.HTTP_REQUEST, url)); // 先
过域名白名单, 不过就抛异常拦下
    return httpClient.get(url); // 通过校验才真正发请求
}

```

看它在干嘛：`@Tool` 注解让 Spring AI 自动把这个方法注册成工具、并根据参数生成 schema；方法体里**第一件事就是过安全校验**，域名不在白名单里直接抛异常、请求根本发不出去；过了校验才真正去请求。安全校验写在执行的第一步，这是硬规矩——这里先用一下这个校验，具体 `Sandbox / SandboxAction` 怎么设计，23、24 节详细展开。

工具怎么汇总、怎么被调。启动时 `ToolRegistry` 把三种来源的工具全扫进来、统一成 `OryxTool`；每个 Agent 启动时再按 Profile 的 `tools` 字段，过滤出自己能用的那批。真正执行时，走的是 17 节讲过的 `ToolExecutor`，流程是这样：



看测试用例。Plugin Tool 写完得能验。一个 Java Plugin Tool 的测试其实很朴素——给输入、调 execute、断言结果，顺便验一下白名单会拦（这两个用例会收编进下面第四部分的 harness）：

```

@Test
void http_get_应能取响应() {
    var result = httpTool.execute(json(
        {"url": "https://api.weather.com/beijing"}));
    assertTrue(result.success());
    assertNotNull(result.content());
}

@Test
void http_get_命中白名单外域名应被拦下() {
    assertThrows(RuntimeException.class, // 23、24 节会讲清楚这里具体抛的是
        SandboxViolationException
        () -> httpTool.execute(json(
            {"url": "https://evil.example.com"})));
}
  
```

第一个用例验“正常能跑通”，第二个验“越界会被拦”。对工具来说，这两条是最该先写的：功能对、且踩不出边界。

有几样先别做。Tool Policy（哪个 Agent 能用哪些工具的 allow/deny 规则）、工具太多时的按需加载、把 OryxOS 自己变成 MCP server 对外暴露、完整的容器级沙箱、一次响应里多个工具并行调，这些都放扩展阶段。核心阶段先用“白名单 + Profile 的 tools 字段限定”顶住。

本节交付物（Spec-Kit 拆解锚点）：

- 代码：OryxTool 接口（含 getInputSchema）、ToolResult、ToolRegistry、AnnotatedToolAdapter、FileTools（3 个）、ShellTools、HttpTools（2 个）、McpClientService、McpToolAdapter；NotifyTools（19 节）在此完成注册；Sandbox 前向接口五件（含 PermissiveSandbox 临时装配，实现本体 24 节）

- 测试：OryxToolContractTest、ToolRegistryTest、FileToolsTest / ShellToolsTest / HttpToolsTest、McpClientServiceTest、McpToolAdapterTest（见验收 harness）
- 配置：.oryxos/mcp_servers.yaml（name/transport/command/env）；Profile 的 tools 字段过滤
- 说明：白名单校验先以接口调用形式接入，Sandbox 本体 23/24 节交付
- 扩展交付（对齐业界主流，见第六部分）：edit_file / grep / glob（扩进 FileTools）、ask_user（InteractionTools + UserInteraction 抽象）、web_search（WebSearchTools + SearchProvider 抽象）

四、验收 harness：把验收标准变成可执行的测试

Tool 体系的 harness 分四块，前三块纯单测，第四块 mock 掉 MCP 连接也不碰网：

测试类	覆盖的验收点
OryxToolContractTest	参数化测试遍历 Registry 里每个工具：name/description/inputSchema 都非空——任何一个工具漏实现 getInputSchema()，这里立刻红（"动手前先检查"那条的自动化版）
ToolRegistryTest	三种来源的工具都以 OryxTool 身份注册进来；按 Profile 的 tools 字段过滤后，子集精确匹配、不多不少
FileToolsTest / ShellToolsTest / HttpToolsTest	各自"正常能跑通 + 越界会被拦"两条（正文里 http_get 那两个用例就是模板）
McpToolAdapterTest / McpClientServiceTest	mock McpClient：listTools 返回的工具被包装注册；execute 转发参数原样、结果包成 ToolResult；连接失败只 WARN、其余工具照常注册、启动不炸

两个最值钱的：

```
@ParameterizedTest
@MethodSource("allRegisteredTools")    // 遍历 ToolRegistry, 新工具自动纳入契约检查
void 每个工具的契约三件套都不能缺(OryxTool tool) {
    assertNotNull(tool.getName());
    assertNotNull(tool.getDescription());
    assertNotNull(tool.getInputSchema());    // 缺了它, Provider 翻译 Function Calling 时直接卡死
}
```

```

@Test
void 某个MCP_server失联_不能拖垮启动和其他工具() {
    when(badClient.listTools()).thenThrow(new ConnectException("refused"));

    mcpClientService.connectAll();           // 不抛异常——外部依赖的可用性不是自己
    的可用性

    assertTrue(toolRegistry.contains("good_mcp_tool")); // 好的 server 照常
    注册
    assertFalse(toolRegistry.contains("bad_mcp_tool"));
}

```

按 Profile 过滤的测试有个"不多不少"的讲究：断言子集**恰好等于**声明列表，多一个（没过滤干净）和少一个（过滤过头）都是错。

五、怎么用，做完怎么验

给 Agent 加工具，对着三档来：

方式一（零代码）：在 `.oryxos/skills/` 写一份 SKILL.md，Profile 里 skills 引用它、`mcp_servers` 引用要复用的 MCP server。

方式二（轻代码）：在 `.oryxos/mcp_servers.yaml` 声明你自己的 MCP server，OryxOS 启动时连上。

方式三（重代码）：写一个带 `@Tool` 注解的 Spring Bean，放进工程，启动自动注册。

用 `oryxos tool list` 能看到当前注册了哪些工具。

harness 全绿后，剩下的人工确认：

- 方式一真跑一次：写一份 SKILL.md + 连一个真实 MCP server，Agent 能读懂意图并调用外部工具完成任务（依赖真模型和真 server，测不了）。
- 方式三真跑一次：`@Tool` 示例工具在 `tool list` 里可见、Agent 能调通。
- 契约三件套、注册过滤、白名单拦截、MCP 失联隔离——已由 harness 覆盖，`mvn test` 绿即打勾。
- 每次工具调用都写进 `tool_invocations`（17 节 `ToolExecutorTest` 已覆盖，这里跑真链路时顺带目检一眼）。

Tool 是 Agent"能干事"的关键。到这一步，配合 Provider、ReAct、CLI，Demo —（每日天气）的对话版（问天气、给穿搭建议）里那个"真的去查天气"的动作，就落地了。

六、把基础工具补齐到业界水准

规划的九个内置工具覆盖了"读写文件、跑命令、调 API、记事、往外推"的最短链路。但对着 Claude Code、Cursor、OpenAI Assistants 这些主流 Agent 产品的公约数一比，会发现少了几样日常任务里高频、缺了就明显笨手笨脚的工具。所以实现时又补了五个——它们不是新概念，全都复用本节已立好的 `OryxTool` 抽象、`@Tool` 注解管道和 `Sandbox` 检查位，正好也是对前面那套设计的一次实战检验：加工具确实只是"多写一个方法 / 多挂一个实现类"。

6.1 缺哪几样，为什么必须补

工具	干什么	不补的话
<code>edit_file</code>	把文件里一段 唯一出现 的旧文本换成新文本	只有整文件覆盖写 (<code>write_file</code>)，改一行配置要把整个文件重写一遍——既费 token 又容易把别的内容冲掉
<code>grep</code>	在路径下按正则搜内容，返回 文件:行号:内容	Agent 想"查一下哪个配置引用了 X"只能一层层 <code>list_dir + read_file</code> 硬翻
<code>glob</code>	按通配 (如 <code>**/*.yaml</code>) 找文件路径	同上，"找出所有 yaml"没有顺手的办法
<code>ask_user</code>	处理中途向用户提问 / 请求确认	危险操作没法找人拍板，human-in-the-loop 没有落点
<code>web_search</code>	用搜索引擎检索网页	日报 / 调研类 Agent 少了"上网查"的第一入口，只能靠业务方各自配 MCP

判断标准还是那句话：这几样是**"基础工具"而非"业务能力"**——任何一个稍复杂的任务都会用到，所以值得进内置清单，而不是丢给 Plugin Tool 每家各写一遍。

6.2 三个文件工具：直接扩进 FileTools

`edit_file` / `grep` / `glob` 本质都是文件操作，跟 `read_file` 同族，所以直接加进 `FileTools` ——同一个 `Sandbox`、同样的"方法第一行过 `enforce`"。三个各有一处值得留意的设计：

- **`edit_file` 要求"旧文本唯一匹配"**：找不到报错、出现多次也报错，逼调用方给足上下文来定位唯一编辑点。这跟 Claude Code / Cursor 的 `edit` 工具是同一套约束——不这么做，"把 0.7 改成 0.8"可能会改错好几处。报错时文件一个字都不动。
- **`grep` / `glob` 有返回条数上限 (200 条)**：超大目录一次全吐出来会把模型的上下文撑爆，超了就截断并注明。
- **`grep` 读到二进制 / 非 UTF-8 文件**：跳过那个文件，而不是中断整次搜索。

6.3 `ask_user`：human-in-the-loop 的落点，接口先行

`ask_user` 有个跟前面工具不一样的地方：它依赖"当前渠道能不能跟人交互"。CLI 场景可以读终端，但定时任务、Web 无状态调用根本没有一个人在另一端等着回答。所以照老规矩——接口先行：

```
public interface UserInteraction {  
    String ask(String question);    // 拿不到回答就抛异常，绝不静默卡住  
}
```

核心阶段挂两档实现：`ConsoleUserInteraction`（读终端的一行）和 `UnsupportedUserInteraction`（无人值守渠道直接报"不支持交互"）。`ask_user` 工具本身只管把问题递给 `UserInteraction`、把回答带回来，至于怎么问、能不能问，交给装配时注入的实现决定。这跟 19 节 `NotifyChannelAdapter`、本节 `Sandbox` 是同一个"接口表达意图、实现按渠道挂"的套路。

一个要想到的点：`ask_user` 读终端和 `chat` 循环读终端是同一个 `stdin`。因为 `ReAct` 循环是同步执行的（原则七），`ask_user` 那一步和对话读行不会真的并发争抢——Agent 问、用户在同一个终端答，一问一答顺序发生。

6.4 `web_search`：同样接口先行，白名单当自己人

`web_search` 走网络，所以跟 `http_get` 一样，第一件事过 `Sandbox` 的 `HTTP_REQUEST` 白名单——不能因为它叫"搜索"就绕过治理。具体用哪个搜索引擎同样抽成接口：

```
public interface SearchProvider {  
    record SearchResult(String title, String url, String snippet) {}  
    List<SearchResult> search(String query);  
}
```

核心阶段挂一档 `DuckDuckGoSearchProvider`（用免 key 的 Instant Answer API），换 Google / Bing / Tavily 只加实现类、不改 `web_search` 工具与调用方。工具本身只负责把结果渲染成模型好读的文本。

6.5 加这五个花了多少代价

值得回头看一眼加这五个工具动了多少地方，因为这正好印证了前面那套抽象的价值：

- 三个文件工具：`FileTools` 里加三个 `@Tool` 方法，没动别的。
- `ask_user` / `web_search`：各加一个工具类 + 一个前向接口 + 一档实现，跟 19 节加 `notify`、24 节加 `Sandbox` 是一模一样的动作。
- 注册：在装配处多 `registerAnnotated` 两行；`Sandbox` 检查位、`tool_invocations` 审计、`Profile tools` 过滤——一行没改，全都自动生效。

这就是"先立统一抽象"省下来的东西：新工具的成本恒定在"写一个类"，不会因为工具越来越多而越加越难。到这里，OryxOS 的基础工具面已经和主流 Agent 产品对齐——再往上就是业务方通过三档 Plugin 接进来的垂直能力了。